

CloudNativePG: A Deep Technical Analysis of the Kubernetes-Native PostgreSQL Operator

TL;DR

- **CloudNativePG (CNPG)** is a Go-based, Apache-2.0-licensed, CNCF Sandbox Kubernetes operator (originally built by EDB, now vendor-neutral) that manages the full lifecycle of primary/standby PostgreSQL clusters by embedding a Go "instance manager" as PID 1 inside each Postgres pod and using the Kubernetes API server itself for state and consensus — deliberately avoiding Patroni, etcd, or Stolon. The current lines are 1.29.x and 1.30.0 (v1.30.0 released June 29, 2026; v1.29.1 released May 8, 2026).
- **Architecture is a split control plane:** a controller-manager Deployment (leader-elected, controller-runtime based) reconciles the `Cluster` CRD and ~10 related CRDs, while each pod's instance manager supervises `postgres`, generates config, runs WAL archive/restore commands, handles promotion/demotion, and reports status back over an HTTPS status endpoint on port 8000 (metrics on 9187). It manages PVCs directly rather than using StatefulSets.
- **Recent releases materially changed the internals:** backup/recovery is being moved out of core into the CNPG-I gRPC plugin interface (Barman Cloud Plugin; in-tree `barmanObjectStore` deprecated since 1.26, removal now targeted for 1.31.0), 1.26 added declarative offline in-place major upgrades via `pg_upgrade --link`, and 1.28 promoted quorum-based failover (with a `FailoverQuorum` CRD) to stable.

Key Findings

What it is. CloudNativePG is an open-source Level V ("Auto Pilot") Kubernetes operator for vanilla PostgreSQL. It was created and open-sourced by EDB under Apache License 2.0, and was officially accepted into the CNCF Sandbox on January 15, 2025 (the CNCF project page lists the acceptance date as January 21, 2025); an earlier application submitted in July 2022 was not accepted. It is now governed by a vendor-neutral community and licensed Apache-2.0 (docs are CC-BY-4.0). Gabriele Bartolini (EDB VP/Chief Architect of Kubernetes) is a founder and the leading maintainer; the initial maintainer group was the top six EDB committers. EDB offers commercial support (Community360) and a downstream commercial product, "EDB Postgres for Kubernetes." The project scope is strict: vanilla Kubernetes + vanilla PostgreSQL only, no forks, not a general-purpose DB operator. [GitHub](#)

Language & stack. The codebase is ~98% Go (`go 1.26.x` in `go.mod`). It is built on `sigs.k8s.io/controller-runtime`, `k8s.io/client-go`, `apimachinery`, `apiextensions-apiserver`, `cli-runtime`, scaffolded with Kubebuilder (evidenced by the `PROJECT` file and layout). The CLI/plugin uses `spf13/cobra`. Postgres connectivity uses `jackc/pgx/v5` and `lib/pq`. Prometheus instrumentation uses `prometheus/client_golang`. CNPG-I plugin communication uses `google.golang.org/grpc` and the `cloudnative-pg/cnpg-i` and `cloudnative-pg/machinery` modules; `cloudnative-pg/barman-cloud` is a dependency. Scheduling uses `robfig/cron`; CSI snapshots via `kubernetes-csi/external-snapshotter/client`.

Binaries. A single `manager` binary (built from `cmd/manager`) serves as both the operator controller-manager and, via subcommands, the in-pod instance manager (`/controller/manager ...`). A separate `kubectl-cnpg` binary (`cmd/kubectl-cnpg`) is the kubectl plugin. PgBouncer poolers run the same manager binary as a supervising sidecar-style process.

Details

1. Role and project status

CloudNativePG introduces the `Cluster` custom resource representing a PostgreSQL cluster of one primary plus optional replicas within a namespace. It automates what a skilled DBA would do manually: bootstrap, failover, switchover, scaling, rolling updates, backup, recovery, configuration, and connection routing via three operator-managed services (`-rw` to the primary, `-ro` to replicas round-robin, `-r` to any instance). The operator continuously updates the `Cluster` `.status` subresource; the same status feeds `kubectl-cnpg status`. It does **not** use StatefulSets — it manages PVCs directly through a "custom pod controller" so it can reattach volumes, reuse storage, and control instance identity/serials. [CloudNativePG](#)

Version state: 90+ releases. v1.30.0 was released June 29, 2026 and adds a `DatabaseRole` CRD and a Lease-based primary-election primitive; v1.29.1 (May 8, 2026) shipped the fix for CVE-2026-44477 (Critical, CVSS 9.4) in the metrics exporter (responsibly disclosed by Mehmet Ince, @mdisec) plus HA data-safety fixes. Supported minor lines track CNCF-supported Kubernetes versions. The default operand image on the 1.29/1.30 line is `ghcr.io/cloudnative-pg/postgresql:18.4-system-trixie` (default PostgreSQL version 18.4).

2. Repository layout (verified against the `main`/v1.29.1 module tree)

- `cmd/manager` — operator + instance-manager entrypoint (`main.go`); `cmd/kubectl-cnpg` — the plugin entrypoint.
- `api/v1` — CRD Go types (`groupName=postgresql.cnpg.io`): `cluster_types.go`, `backup_types.go`, `scheduledbackup_types.go`, `pooler_types.go`, `database_types.go`, `publication_types.go`, `subscription_types.go`, `imagecatalog_types.go` (ImageCatalog + ClusterImageCatalog), `failoverquorum_types.go`, plus `zz_generated.deepcopy.go`. Ten Kinds total.

- `internal/cmd/manager` — subcommands: `controller` (runs the operator; `RunController` in `controller.go`/`cmd.go`), `instance` (with `initdb`, `join`, `pgbasebackup`, `restore`, `restoresnapshot`, `run`, `status`, `upgrade/{execute,prepare}`), `backup`, `bootstrap`, `walarchive`, `walrestore`, `pgbouncer`, `show`, `debug`.
- `internal/controller` — operator-side reconcilers: `cluster_controller.go`, `backup_controller.go`, `scheduledbackup_controller.go`, `pooler_controller.go`, `plugin_controller.go`, plus reconcilers for the newer CRDs, and a `rollout` subpackage.
- `internal/management/controller` — instance-side reconcilers: `instance_controller.go`, plus `roles`, `slots/{infrastructure,reconciler,runner}`, `tablespaces`, `externalservers`.
- `internal/webhook/v1` — admission webhooks: `cluster_webhook.go` (+ `backup`, `scheduledbackup`, `pooler`), setup funcs like `SetupClusterWebhookWithManager`.
- `pkg/management/postgres` — core PostgreSQL lifecycle: `Instance` type (`instance.go`), `join.go`, `restore.go`, `backup.go`, `bootstrap/initdb`, plus subpackages `archiver`, `logpipe` (CSV→JSON), `pool`, `metrics`, and `webserver` (with `probes`, `metricserver`, and `client/{local,remote,common}`).
- `pkg/reconciler` — `persistentvolumeclaim`, `instance/{certificate,storage}`, `backup/volumesnapshot`, `hibernation`, `majorupgrade`, `replicaclusterswitch`, `podselector`.
- `pkg/certs` — operator PKI (CA + webhook cert). Other `pkg` subpackages: `postgres` (`config`, `hba`, `replication`, `plugin`), `specs`, `utils`, `versions`, `promotiontoken`, `podspec`, `configfile`.
- `internal/cnpi/plugin/{client,connection,operatorclient,repository}` — CNPG-I plugin client machinery. `config/` — kustomize/CRD manifests; `releases/` — flattened install YAML; `docs/src/` — the documentation source; `tests/e2e` — Ginkgo/Gomega end-to-end tests.

3. Architecture

Operator (control plane). Deployed (default) in `cnpg-system` as the `cnpg-controller-manager` Deployment, normally one replica, using controller-runtime's manager with leader election (LeaderElectionID `db9c8771.cnpg.io`) so multiple replicas can run active/passive. The manager wires reconcilers for `Cluster`, `Backup`, `ScheduledBackup`, `Pooler`, plugins, and the newer resources, each with configurable concurrency (`-max-concurrent-reconciles`, default 10). It exposes metrics on 8080 and a webhook server on 9443. Because state lives in the Kubernetes API (the `Cluster` `.status`, annotations, and — in 1.30 — a Lease object acting as a primary-election mutex), there is no etcd/Consul/ZooKeeper dependency beyond Kubernetes itself: **the API server is the DCS.** [CloudNativePG](#)

Instance manager (data plane). Each Postgres pod runs the same manager binary as **PID 1** (`instance run`), which forks/adopts the `postmaster`. It: backs the startup/liveness/readiness probes (startup default `pg_isready`, optionally `query` or `streaming`-lag gated); generates `postgresql.conf`/`custom.conf`/`override.conf` and `pg_hba.conf`; sets `archive_command = '/controller/manager wal-archive ... %p'` and `restore_command = '/controller/manager wal-restore ... %f %p'`; performs promotion/demotion/`pg_rewrite`; and serves an HTTP(S) status/control endpoint on **port 8000** (`/pg/status`, `/readyz`) plus the Prometheus exporter on **port 9187** (`/metrics`). The operator scrapes `:8000/pg/status` from each pod each reconcile to gather LSNs, timeline, replication state, and role. Graceful shutdown is staged: smart shutdown (up to `smartShutdownTimeout`, default 180s) → fast shutdown → wait for WAL archiving/streaming (up to `stopDelay`, default 1800s) → forced.

Security of operator↔instance channel. Historically the status port relied on network isolation; a hardening introduced in CNPG 1.30.0 (not backported, addressing GHSA-7qwx-x8ff-3px9) has the operator generate an in-memory ECDSA P-256 client certificate at startup and reconcile its SHA-256 fingerprint into the cluster status, so the instance manager rejects requests to sensitive endpoints that do not present a matching certificate. Earlier releases should continue to restrict the status port with a NetworkPolicy. The instance-manager in-place upgrade endpoint is SHA-256-pinned.

Admission webhooks. Both mutating (`ClusterCustomDefaulter`, applies defaults) and validating (`ClusterCustomValidator`, plus `Backup/ScheduledBackup/Pooler` validators) webhooks are served at `/mutate-postgresql-cnpg-io-v1-*` and `/validate-postgresql-cnpg-io-v1-*`. Since 1.30, defaulting/validation also runs as a reconciliation fallback when webhooks are unavailable, and CEL rules enforce immutability (e.g., `Cluster` reference immutable on `Database/Pooler/Publication/Subscription/ScheduledBackup`). [GitHub](#)

4. CRDs and API (`postgresql.cnpg.io/v1`)

`Cluster` (the core resource), `Backup`, `ScheduledBackup`, `Pooler` (PgBouncer), `ImageCatalog` and `ClusterImageCatalog` (namespaced/cluster-scoped major-version→image maps, extended in 1.29 to carry extension images), `Database`, `Publication`, `Subscription` (declarative databases and logical replication), and `FailoverQuorum` (tracks synchronous-replication quorum state, created by the controller when quorum failover is enabled). The `barmancloud.cnpg.io/v1` `ObjectStore` CRD is defined by the Barman Cloud Plugin, not core CNPG. 1.30 adds `DatabaseRole`. [GitHub](#) [cloudnative-pg](#)

5. Implemented capabilities

- **Bootstrap:** `initdb` (fresh cluster), `recovery` (restore a physical base backup + replay WAL, optionally to a PITR target, via `externalClusters`), and `pg_basebackup` (clone another live instance over streaming replication). `recovery/pg_basebackup` can also seed replica clusters.
- **HA / replication:** async and synchronous streaming replication, both **priority-based and quorum-based** synchronous (`.spec.postgresql.synchronous` with `method: any/first`, `number`, `dataDurability`), managed replication slots for all replicas, user-defined slot synchronization, and logical-decoding failover (native `sync_replication_slots` on PG17+, `pg_failover_slots` earlier). `wal_level` defaults to `logical`.
- **Automated failover** with optional delay, and **quorum-based failover** (1.28 stable) implementing a Dynamo-style $R + W > N$ promotable-replica check before promotion.
- **Controlled switchover**, including cross-region replica-cluster switchover via demotion/promotion tokens (no re-clone).
- **Rolling updates** for minor PG versions and operator upgrades, with `primaryUpdateStrategy` (`unsupervised`/`supervised`) and `primaryUpdateMethod` (`restart`/`switchover`); optional in-place instance-manager updates (`ENABLE_INSTANCE_MANAGER_INPLACE_UPDATES`) that adopt the running postmaster without a switchover. `CloudNativePG`
- **Backup/recovery:** WAL archiving + base backups to object stores (legacy in-tree `barmanObjectStore`, deprecated since 1.26; now via the **Barman Cloud CNPG-I plugin** which runs as a sidecar), **volume snapshots** (CSI, hot or cold/fenced), full and PITR recovery, scheduled + on-demand backups, backup-from-standby, parallel WAL archive/restore.
- **Replica clusters / distributed topology** across Kubernetes clusters (streaming and/or WAL-shipping), delayed replicas, cascading.
- **Connection pooling** via the `Pooler` CRD (PgBouncer Deployment, `type: rw|ro`, `session|transaction` mode, `auth_user`/`auth_query` with a `cnpg_pooler_pgbouncer` role, exporter on 9127).
- **Monitoring** (built-in exporter on 9187, custom SQL queries via ConfigMap/Secret), **declarative** management of roles, databases, extensions, schemas, FDWs, tablespaces, publications/subscriptions, **fencing**, **declarative hibernation**, **TLS/cert-manager** integration, **storage** (separate WAL volume, tablespace volumes, online volume expansion), and **offline in-place major upgrades** via `pg_upgrade`.
- **CNPG-I:** a gRPC plugin interface (inspired by CSI) letting external plugins hook reconciliation (`ReconcilerHooks`), lifecycle hooks, backup/WAL services) without forking core. `GitHub`

6. Detailed workflow pathways

(a) Cluster creation. User applies a `Cluster` CR → mutating webhook applies defaults, validating webhook checks the spec → `ClusterReconciler.Reconcile` runs. The operator creates the CA/TLS secrets, the three services, and PVCs (`pkg/reconciler/persistentvolumeclaim`). For the first instance it runs `createPrimaryInstance`, which creates a Kubernetes **Job** that runs `manager instance initdb` to build PGDATA (and a separate WAL PVC if `.spec.walStorage`), then starts the primary pod (instance manager as PID 1). Once the primary is `Ready`, the operator scales out: for each replica it creates a join Job running `manager instance join`, which performs `pg_basebackup` from the primary, configures `primary_conninfo`/`restore_command`, and starts the pod as a hot standby. Status converges to "Cluster in healthy state."

(b) Cluster reconciliation loop. Each iteration: read the `Cluster` spec + child resources; poll every pod's instance manager at `:8000/pg/status` for role, LSNs, timeline, WAL-receiver state; compute desired vs. observed; then reconcile services, PVCs, secrets/certs, PodDisruptionBudgets, and pods; decide on rollouts (`isPodNeedingRollout`), failover, or scaling; write the aggregated status back to the `.status` subresource and `instancesReportedState`. The `cnpg.io/reconciliationLoop: disabled` annotation can pause it for emergencies.

(c) Failover. When the primary's readiness probe fails (or, for node loss, its `Ready` condition is flipped to `False` by the node controller ~40-55s after the node becomes unreachable), the controller enters a **two-phase** procedure: (1) mark `targetPrimary` `pending`, forcing the old primary to shut down so replica WAL receivers stop (prevents timeline divergence); the failing primary attempts fast shutdown (timeout `switchoverDelay`) then immediate shutdown. (2) After WAL receivers stop, elect the most-aligned replica (highest LSN among ready/promotable instances), which promotes itself; other replicas re-point via `primary_conninfo` and use `pg_rewrite` where needed; the demoted old primary rejoins as a standby. With quorum failover enabled, promotion is gated on the $R + W > N$ check against the `FailoverQuorum` object; if unsatisfied the operator refuses to promote (manual `kubectl cnpg promote` override exists). A 1.30 fix labels the old primary `unhealthy` during failover so the `-rw` service never routes back to it.

(d) Switchover (planned). The operator picks/uses a target replica, gracefully demotes the current primary (checkpoint + clean shutdown so no data loss), promotes the target, updates the `-rw` service endpoint, and the former primary follows the new timeline (via `pg_rewrite` if required). Triggered manually (`kubectl cnpg promote`) or automatically by config changes/operator upgrades when `primaryUpdateMethod: switchover`.

(e) Backup & recovery. *Object-store path:* `archive_command` on the primary calls `manager wal-archive`, which (via the Barman Cloud plugin sidecar or legacy in-tree Barman) ships WAL (parallelizable) to S3/Azure/GCS; a `Backup`/`ScheduledBackup` triggers `barman-cloud-backup` producing a base-backup tarball (default target = most-aligned replica). *Volume-snapshot path:* CSI `VolumeSnapshot` of the PGDATA

(and WAL) PVCs, hot or cold (fencing the standby). *Recovery* is never in-place: a new `Cluster` with `bootstrap.recovery` referencing an `externalClusters`/ObjectStore source spins up an init container that runs `barman-cloud-restore` for the base backup, then PostgreSQL replays WAL via `restore_command` (`manager wal-restore`) (parallelizable) up to the target or latest; the operator then sets the superuser password, starts the new primary, and joins replicas. An "expected empty archive" safety check prevents clobbering an existing WAL store.

(f) Rolling update. Triggered by `imageName`/catalog/extension changes, restart-requiring config changes, `resources` changes, or operator upgrade. The operator updates replicas one at a time, **highest serial first**, keeping the same PVCs/identity (pods deleted and recreated with the new image), moving service endpoints away from the updating pod; the **primary is updated last** via the chosen `primaryUpdateMethod`. Operator upgrade is two-step: upgrade the controller + CRDs, then the instance manager in every pod (rolling update, or in-place if enabled).

(g) Instance-manager lifecycle. As PID 1 it starts/adopts `postgres`, keeps config files reconciled to the desired `Cluster` state (it reacts to config-map/secret changes labeled `cnpg.io/reload`), backs the probes, executes WAL archive/restore commands, performs promotion/demotion and `pg_rewrite`, runs the declarative role/database/slot/tablespace reconcilers locally, streams Postgres CSV logs re-emitted as JSON on stdout, exports metrics on 9187, and reports status to the operator on 8000. On SIGTERM it runs the staged shutdown.

`CloudNativePG`

7. Major-version upgrades (1.26+)

Declarative offline **in-place** major upgrade: bump the operand image major version in the spec → operator records `.status.pgDataImageInfo`, shuts down all pods (smart), creates new PGDATA/WAL/tablespace directories, runs `pg_upgrade --check` then `pg_upgrade --link` (hard links → time scales with number of relations, not data size), swaps directories on success, then destroys and recreates replicas. Requires same-OS-distro images; downtime required; failure needs manual rollback (revert image, delete `-major-upgrade` Job). Blue/green alternatives exist via logical import (`pg_dump`)/(`pg_restore`) and native logical replication (`Subscription`), the latter enabling near-zero-downtime online upgrades. Note that CNPG does not run `VACUUM ANALYZE` afterward — statistics must be rebuilt manually.

8. CNPG-I plugin architecture

A gRPC protocol (protobuf, `google.golang.org/grpc`) modeled on CSI. Plugins register either as a **sidecar** in the operator Deployment (exposing gRPC over a Unix socket in a shared `/plugins` volume, `PLUGIN_SOCKET_DIR`) or as a **standalone Deployment** (TCP gRPC behind a Service with mTLS, annotations `cnpg.io/pluginPort`, `cnpg.io/pluginClientSecret/ServerSecret`); standalone plugins can be discovered dynamically via labeled Services, sidecars require an operator restart). Hook surfaces include reconciler hooks (`ReconcilerHooks`), per-Kind (`KIND_CLUSTER`)/(`KIND_BACKUP`), with `CONTINUE`/`REQUEUE`/`TERMINATE` behaviors), lifecycle hooks, pre/post reconcile, backup and WAL services, and a Postgres interface. WAL archiving is delegated to whichever plugin sets `isWALArchiver: true`. The **Barman Cloud Plugin** is the reference/flagship, running a sidecar next to each instance (or standby for offloaded backups) using a dedicated image containing barman-cloud, driven by the `ObjectStore` CRD. `Go Packages`

Recommendations

- **For deep code study, start at:** `cmd/manager/main.go` → `internal/cmd/manager/controller/controller.go` (`RunController`, manager wiring, leader election) → `internal/controller/cluster_controller.go` (operator reconcile) → `internal/management/controller/instance_controller.go` (the in-pod `InstanceReconciler`, which contains the HA/failover logic: `verifyPgDataCoherenceForPrimary`, `reconcileOldPrimary`, `updateFailoverQuorumObject`) → `pkg/management/postgres/{instance.go, join.go, restore.go}` for `Startup`/`Shutdown`/`Demote`/`Rewind`/`PromoteAndWait`. Read `pkg/reconciler/persistentvolumeclaim` to understand the no-StatefulSet PVC model, and `pkg/management/postgres/webserver` for the 8000/9187 endpoints.
- **For a hands-on architecture mental model**, deploy 1.29/1.30 on kind with a 3-instance cluster, enable `synchronous` + `failoverQuorum: true`, and watch `kubectl cnpg status` while draining the primary's node to observe the two-phase failover and quorum gating.
- **Adopt the plugin-based backup path now:** because in-tree `barmanObjectStore` is deprecated (since 1.26) and now slated for removal in 1.31.0 (per the 1.30.0 release notes), new deployments should use the Barman Cloud Plugin (+ `ObjectStore` CRD) or CSI volume snapshots. Benchmark restore/RTO with `maxParallel` WAL restore tuned.
- **Benchmarks/thresholds that change decisions:** use quorum failover only when you can keep $R + W > N$ after a single failure (i.e., avoid 2-node sync topologies); prefer `primaryUpdateMethod: switchover` when image pull time is significant (promoted node already runs the target image); reserve dedicated `node-role.kubernetes.io/postgres` nodes and deploy in multiples of three across AZs for shared-nothing HA.

Caveats

- **This report reflects the `main`/1.29–1.30 line as of July 2026.** Because backup/recovery is mid-migration to CNPG-I, behavior differs sharply by version: in-tree Barman works through 1.30 but is deprecated and now targeted for removal in 1.31.0; some capabilities

(quorum failover 1.28, offline major upgrade 1.26, image-catalog extensions 1.29, Lease-based primary election and ECDSA operator→instance auth 1.30) do not exist in older minors. Verify against the exact operator version in use.

- **Cross-Kubernetes-cluster failover is out of scope:** the operator only acts within one Kubernetes cluster; distributed-topology promotion/demotion across clusters must be driven manually or by a higher-level tool.
- **Source filenames:** directory/package structure and CRD Kinds were verified against the module index and source; a few individual reconciler/`(*_types.go)` filenames follow Kubebuilder's standard 1:1 naming convention and were inferred rather than each independently opened. GitHub blocks automated scraping of its tree pages, so the map leans on the pkg.go.dev module mirror (v1.29.1) plus verified source citations.
- Some numeric defaults (probe timeouts, `(node-monitor-grace-period)` timings, shutdown delays) are Kubernetes/CNPG defaults that operators frequently tune; treat them as starting points, not guarantees.